

LLM platform migration plan

Owner: PA Office backend

Status: Parked — approved after field research on the current QR + AI Uploads flow

Author: Engineering

Date: 2026-07-05

Roadmap for moving off the Google AI Studio Developer API onto Vertex AI, and the honest path from there toward operating our own LLM. Captures the current state, the three progressive stops, the numbers, and the operational constraints so we can pick this back up cold.

1. Where we are today

The three Gemini-backed services in the codebase all instantiate the unified `google-genai` SDK against the Developer API surface:

- `backend/src/services/summarisation.py` — QR / form petition summariser
- `backend/src/services/petition_extraction.py` — AI Uploads scanned-doc reader
- `backend/src/services/stt_service.py` — audio → text before summarisation

Client construction is identical in all three:

```
from google import genai
client = genai.Client(api_key=settings.GEMINI_API_KEY)
```

That single line is the whole Google AI Studio surface. Everything downstream (`generate_content`, `response_schema`, `system_instruction`, service tier, fallback chain) is portable.

This is good for prototyping. It is not the right surface for a Minister's office running at production scale carrying citizen PII.

2. Why move — the three things that actually matter

2.1 Auth model — a bearer API key is a weak boundary

Today the entire office's Gemini capability hangs on `GEMINI_API_KEY` sitting in Railway env + `backend/.env`. Anyone with that string can call anything the key covers. Leaked once → revoke, regenerate, rotate everywhere.

Vertex AI uses Google IAM: a service account with a specific role (`roles/aiplatform.user`) authenticated via Application Default Credentials or Workload Identity. Roles are audited centrally, keys can be scoped and short-lived, and every call is attributable to a principal.

2.2 Data governance — the paid AI Studio tier is acceptable, Vertex is defensible

Surface	Trains on your data?	Logging	Controls
AI Studio free tier	YES — used for model training + human review	Retained	None. Never send PII.
AI Studio paid tier	No	Transient, abuse-only	None (region-agnostic)
Vertex AI	No	Transient, abuse-only	CMEK, VPC-SC, zero-data-retention (via account team), Cloud Audit Logs, HIPAA-compatible workloads

We are on the paid tier so we are technically safe, but "trust us" is not the same as "prove it". When someone in the CM's office asks whether petitions touch the internet, Vertex + CMEK + VPC-SC lets us point at contract clauses and audit logs. Paid-tier AI Studio lets us shrug and quote the ToS.

2.3 Data residency — Chennai → Mumbai vs Chennai → Iowa

- **AI Studio:** routes through Google's global infrastructure, no regional pinning available.
 - **Vertex asia-south1 (Mumbai):** Gemini 2.5 Flash live today. Data stays in India. Latency from Chennai ≈ **40–60 ms** to Mumbai vs 200–300 ms transatlantic. Faster **and** legally cleaner.
- For a government workload this is the single strongest argument. It also happens to be the free performance win.

3. What does NOT change — the good news

- **Cost is essentially the same.** Standard pay-as-you-go rates are identical for the same model on paid AI Studio and Vertex Standard.
- **The prompts we shipped this week transfer with zero changes** — same models, same JSON schema surface, same system_instruction.
- **The eval, the fallback chain, the retry logic** — all reused as-is.

Standard PayGo pricing snapshot (USD, per 1M tokens)

Model	Input (text/img/video)	Output
gemini-2.5-flash	\$0.30	\$2.50
gemini-2.5-flash-lite	\$0.10	\$0.40
gemini-2.5-pro	\$1.25 (≤200K) / \$2.50 (>200K)	\$10 / \$15
gemini-3.5-flash	\$1.50	\$9.00
gemini-3.1-flash-lite	\$0.25	\$1.50

Model	Input (text/img/video)	Output
gemini-3.1-pro (preview)	\$2.00	\$12.00

Same table on both surfaces. Where costs **do** differ:

- **Vertex Batch mode** — 50% off list, ≤24 h turnaround. Useful for nightly

eval re-runs and reporting jobs, not for real-time QR summarisation.

- **Vertex Provisioned Throughput** — monthly commit via sales team, gives you reserved capacity and 99.9% SLA. Only makes sense at thousands of calls/hour sustained. **We are nowhere near this yet.**

- **Vertex non-global endpoint** (e.g. asia-south1 pin) — ~10% premium over the global endpoint. Worth paying for residency.

Bottom line: this is not a savings play. It's a governance play with a free latency improvement.

4. Migration mechanics

4.1 Code change

The unified google-genai SDK already installed ($\geq 2.8.0$) drives both surfaces. Client construction is the only line that changes:

```
# TODAY (AI Studio Developer API):
client = genai.Client(api_key=settings.GEMINI_API_KEY)

# AFTER (Vertex AI, Mumbai):
client = genai.Client(
    vertexai=True,
    project="tn-pa-office",      # GCP project id
    location="asia-south1",     # Mumbai
)
```

Wrap it behind a settings flag so we can toggle back instantly if something misbehaves:

```
if settings.USE_VERTEX_AI:
    client = genai.Client(
        vertexai=True,
        project=settings.GCP_PROJECT_ID,
        location=settings.GCP_LOCATION,
    )
else:
    client = genai.Client(api_key=settings.GEMINI_API_KEY)
```

4.2 One-time GCP setup

1. Create GCP project + attach billing account
2. Enable `aiplatform.googleapis.com`
3. Create a service account (e.g. `vpa-pilot-backend@...`) with role

`roles/aiplatform.user`

4. Download JSON key **OR** use Workload Identity if the VPS supports it
5. Add `GOOGLE_APPLICATION_CREDENTIALS=/etc/vpa/gcp-key.json` to `systemd env`
6. Add `USE_VERTEX_AI`, `GCP_PROJECT_ID`, `GCP_LOCATION` to `settings`

4.3 Verification steps before cutover

- Run `backend/eval/run_eval.py` on the full 19-case set against Vertex

(`gemini-2.5-flash`, `asia-south1`). Compare ministry / category / latency vs the baseline we have on `main`.

- Fire one QR petition end-to-end in preview with `USE_VERTEX_AI=true`.
- Fire one AI Uploads scan end-to-end. Confirm strict extraction still fires.
- Verify audit log entries appear in Cloud Logging with the service account

attribution.

4.4 Rollback

Flip `USE_VERTEX_AI=false`, restart the backend. Instant.

4.5 Effort

One working day, plus roughly a week of soak time watching Cloud Logging before we declare cutover done.

5. The "own LLM" path — three steps, do not skip

Stop 1: Fine-tune Gemini on our petition corpus (Vertex tuning)

Still uses Google, still uses Gemini, but on a model version fine-tuned to our handwriting patterns, Tamil script quirks, and TN-specific ministry mappings.

- **What:** Vertex Supervised Fine-Tuning on `gemini-2.5-flash-tuning`. Upload

labelled examples (petition → correct ministry / category / summary), get a customized model version pinned to our project.

- **When:** after we have ~500–1000 human-reviewed petitions. Right now we have 19 gold cases. **This is a Q4 2026 conversation, not now.**

- **Cost:** ~\$8 / 1M training tokens + a per-hour deployment charge for the tuned endpoint. Training a first round on ~1000 petitions is probably under \$100. Endpoint costs scale with QPS.

- **Prep now:** log every PA override (ministry/category/summary/name correction) into a `training_candidates` table. Costs nothing today, saves months later when we want the corpus.

Stop 2: Vertex-hosted open-weight models (Gemma / Llama / Mistral / Qwen)

Break the Google-Gemini dependency but stay on Google-managed GPUs.

- **What:** Vertex Model Garden lets us deploy Gemma 3, Llama 3.3, Mistral, Qwen 2.5 etc. on a managed endpoint. Same google-genai-style API surface.
- **Cost:** per-hour endpoint charges, not per-token. L40S $\approx$ \$2–4/hr, A100 $\approx$ \$4–7/hr. Only cheaper than Gemini API at very high sustained volume.
- **When:** only if we have a specific reason to avoid Gemini (policy shift, vendor risk, licensing). For a school-ed Minister's office we have no such reason yet.

Stop 3: Self-host on our own GPUs — full sovereignty

Petition data never leaves our infrastructure. Real cost, real ops burden.

- **What:** quantized Llama 3.3 70B on a single A100 (80 GB), or Gemma 3 27B on 2 $\times$ L40S, or Qwen 2.5 32B on a single L40S. All three cover Tamil well.
- **Where:**
 - Cloud rental: RunPod / Lambda / Modal — L40S $\approx$ \$1.50–2.50/hr, A100 $\approx$ \$2–4/hr on demand.
 - On-prem: TN State Data Centre, or Dell/Supermicro with 2 $\times$ RTX 6000 Ada (~\$20L capex).
- **Cost math:** $24 \times 30 = 720$ h/mo $\times$ \$2/hr $\approx$ **\$1,440/mo** on-demand cloud.

That is ~10–30 $\times$ Vertex costs at our current volume. Only wins if we cross into millions of calls/month, **or** if "data never leaves the building" becomes a hard requirement (CM mandate, national security tier).

- **Quality gap:** Llama 3.3 70B and Gemma 3 27B are within ~5–10% of gemini-2.5-flash on standard English tasks. Gap on **Tamil handwriting OCR** is larger — Gemini's multimodal training is currently ahead of open-weight models for that specific need. Vision is where we would take the biggest hit.
- **Ops burden:** model updates, GPU driver churn, availability engineering, quantization tuning. Not a spare-time job. Realistically requires an MLOps engineer on payroll.

6. Recommendation

Do all four in order. Do not skip.

Now (this month) — migrate to Vertex asia-south1

- One day of engineering.
- Cost neutral.
- Latency improves.
- Data stays in India.
- IAM replaces the bearer key.

- Audit logs come free.
- Governance story becomes defensible.

Q3 2026 — start the training corpus

- Log every PA override into `training_candidates`. Zero-cost background prep.

Q4 2026 — Vertex SFT of gemini-2.5-flash

- Only after ~1000 corrections logged.
- Ship the tuned model if eval beats base.

2027+ — Self-host, only if sovereignty is mandated

- Otherwise stay on Vertex.

7. Open decisions before we start Step 1

1. **GCP project ownership** — new project just for the backend, or piggyback on an existing TN eGovernance project?

2. **Billing** — who is the paying entity? (The Minister's office directly, a TN eGovernance vote, a vendor account?)

3. **Key strategy** — Application Default Credentials via service account key file on the VPS, or set up Workload Identity Federation (harder, safer)?

4. **CMEK now or later?** — Vertex works fine without customer-managed keys.

Turning on CMEK requires a KMS keyring and adds one more thing to lose. Recommendation: **defer** until governance actually asks for it.

5. **VPC-SC now or later?** — Same argument. Big win when the audit team asks, overhead until then.

8. What blocks starting

- Provisioning a GCP project + billing (procurement)
- Answers to the five decisions in Section 7

Once those are in, the engineering work is a single focused day.